

Linguagens e Programação

Exame Época Normal**3 de julho 2023****N.º** _____ **Nome:** _____

- Responda a **cada grupo em folhas separadas**. Deve entregar, pelo menos, **uma folha por grupo**
- Indique o **seu número e nome** em cada folha que entregar
- A **prova é com consulta** (1 folha A4) e tem a duração de 2 horas

Grupo I (5 valores)

Em cada pergunta só há uma resposta certa, se marcar mais do que uma será considerado que não respondeu à questão.

- Responda na própria folha, marcando com um X ou ■ (pode usar o lápis)
- Se pretender anular uma resposta escreva “anulada” à esquerda da mesma (por exemplo, anulada ■-xpto)

- Um compilador é um tradutor de linguagens que:
 - ☐ traduz uma linguagem gerando os símbolos terminais
 - ☐ junta as bibliotecas ao respetivo programa fonte
 - ☐ traduz um programa escrito em linguagem fonte em código alvo
 - ☐ compila uma linguagem juntando os símbolos terminais
- A função de um analisador léxico consiste em:
 - ☐ gerar sequências de tokens a partir do código fonte
 - ☐ produzir código fonte
 - ☐ ler os caracteres do programa-fonte e a sintaxe da frase
 - ☐ fazer o ‘parse’ do programa-fonte
- O conflito “Shift - Reduce” acontece quando:
 - ☐ para um dado *token* retornado pelo analisador léxico, duas ou mais regras gramaticais podem ser usadas realizando um **Reduce**
 - ☐ para um dado token retornado pelo analisador léxico, duas ou mais regras podem ser aplicadas realizando pelo menos um **Shift** ou um **Reduce**
 - ☐ uma regra pode, consumindo o próximo token, progredir numa produção da gramática sem a completar
 - ☐ para um dado token retornado pelo analisador léxico, o analisador não sabe se deve realizar uma ação de **Shift** ou **Reduce**.
- Considere a gramática $G = (\{S\}, \{a, b\}, P, S)$, onde P é: $S \rightarrow aSbS \mid bSaS \mid \varepsilon$. Não é possível construir um analisador sintático preditivo para esta gramática porque:
 - ☐ G não é uma gramática regular
 - ☐ G é uma gramática $LL(1)$
 - ☐ G é uma gramática recursiva à esquerda
 - ☐ Nenhuma das anteriores
- A metodologia de análise sintática seguida pelo ANTLR é designada por:
 - ☐ Análise sintática ascendente
 - ☐ Análise sintática descendente
 - ☐ Análise sintática recursiva
 - ☐ Todas as anteriores

Grupo II (5 valores)

1. [1,5 val.] Escreva uma expressão regular que aceite frases cujas linhas têm a seguinte estrutura em formato EBNF:

{<título>;<duração>;<palco>[;<convidado>]}

em que o formato de cada campo é o seguinte:

- **<título>** - string composta por caracteres alfanuméricos ou espaço em branco, sendo que o primeiro e o último nunca podem ser espaços em branco e o número total de caracteres varia entre 2 e 40;
- **<duração>** - duração máxima de 20 minutos (20:00), no formato **MM:SS**, em que **MM** assume valores de **00** a **20** e **SS** assume valores de **00** a **59**;
- **<palco>** - uma das seguintes opções: **PA, PB, PC**;
- **<convidado>** - string composta por caracteres alfanuméricos ou espaço em branco, com tamanho superior a 2;

Exemplo:

```
Intro;00:45;PA
Exito de abertura;03:30;PA
Piano;05:00;PB;Fa convidado
Balada;10:15;PC;Cantor convidado 1;Cantor convidado 2
```

2. Considere as seguintes expressões regulares (ER):

R1 = 2(2|1*)(0?|1?)(0|1)

R2 = (22|2|21+)(0+|1+)

- a) [0,5 val.] Descreva informalmente a linguagem gerada pela expressão regular **R1**.
b) [0,5 val.] Mostre que as ER **R1** e **R2** não geram a mesma linguagem.

3. Considere a seguinte linguagem e alfabeto $\Sigma = \{z, x\}$:

L = {w ∈ Σ* | w contém uma substring zz ou xx ou ambas}

- a) [1 val.] Defina formalmente um AF que reconheça a linguagem **L**, considerando o tuplo **A = (S, Σ, S0, F, δ)**.
b) [1,5 val.] Minimize o autómato finito resultante da alínea a), usando o método formal de minimização. Justifique todas as decisões e pressupostos que assumir. Represente graficamente o autómato finito obtido.

Grupo III (5 valores)

1. Considere a seguinte gramática **G**:

$Z \rightarrow AaBcA \mid AbBcA$

$A \rightarrow aA \mid \epsilon$

$B \rightarrow bB \mid b$

- [0,5 val.] Defina formalmente a gramática **G**.
- [1 val.] Classifique a gramática **G**, segundo a hierarquia de Chomsky. Justifique.
- [1,5 val.] A gramática é ambígua? Justifique.

2. [2 val.] Escreva uma gramática que represente a linguagem definida pelo autómato finito **B** representado na Tabela 1.

Tabela 1 – Autómato Finito **B**

	x	y
$\rightarrow S_0$	$\{S_0, S_1\}$	$\{S_1\}$
$*S_1$	$\{S_1, S_2\}$	$\{S_0, S_1\}$
$*S_2$	$\{S_2\}$	$\emptyset$

Grupo IV (5 valores)

A empresa **ViaDigital** pretende analisar a quantidade de acessos efetuados ao servidor, diariamente. Para tal, desenvolveu um processo automático que recebe a informação em ficheiros de texto, cujas linhas têm a seguinte estrutura em formato EBNF (em que os campos opcionais são apresentados entre `[]` e os que se repetem 0 ou mais vezes entre `{ }`):

`<user>;<function>;<login>,<duration>,{<login>,<duration>,<email>}`

O formato de cada campo é o seguinte:

- `<user>` – String iniciada com letra maiúscula, mínimo 4 e máximo 8 carateres;
- `<function>` – Uma das seguintes strings: `ADMINISTRADOR`, `COMPRAS`, `DISTRIBUICAO`, `COMERCIAL` e `USER`;
- `<login>` – String no formato `HH:MM` representando uma grandeza horária válida;
- `<duration>` – Inteiro entre 1 e 100;
- `<email>` – String iniciada com letra minúscula, seguida de letras minúsculas ou números (máximo 10 caracteres), seguido de `@viadigital.com`.
- Cada **user** pode efetuar vários logins por dia.

```
%{
#include "grupoiv.tab.h"
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <math.h>
}%
%option nounput
%option noinput
%option noyywrap
%%
[A-Z][a-z0-9]{3,7}          yyval.id=strdup(yytext); return USER;
(ADMINISTRADOR|COMPRAS|DISTRIBUICAO|COMERCIAL|USER) return FUNCTION;
([0-9]|[1-9][0-9]|100)      yyval.inteiro=atoi(yytext); return DURATION;
([0-1][0-9]|2[0-3]):[0-5][0-9] yyval.id=yytext;return LOGIN;
[a-z][a-z0-9]{1,9}@viadigital\.com return EMAIL;
", "                          return yytext[0];
"; "                          return yytext[0];
[ \t]                          ;
.                              printf ("Erro%s\n",yytext);
\n                             return yytext[0];
%%
```

Considere o **Flex** definido na tabela, defina a gramática e crie utilizando o **Bison** um programa que:

- Reconheça a validade do ficheiro de texto;
- Produza resultados com o seguinte formato:

O <user> fez <Nlogins> logins acumulando um total de <Nminutos> minutos.\n

No final, deve apresentar o utilizador com a maior quantidade de logins efetuados. Para o caso de existir mais do que um, deve identificar apenas o primeiro.

Exemplo:

Ficheiro de entrada:

```
User123;ADMINISTRADOR;03:14,20,12:15,90,21:30,10, user123@Viadigital.com
User753;USER;03:14,2,12:20,5,21:00,8, user753@Viadigital.com
User519;COMPRAS;09:11,5,12:20,90,21:30,10,23:45,6,user519@Viadigital.com
User321;DISTRIBUICAO;10:20,90,12:10,99, user321@Viadigital.com
User654;ADMINISTRADOR;12:15,90,21:30,10, user654@Viadigital.com
User987;COMERCIAL;03:15,25, user987@viadigital.com
```

Resultado produzido:

```
O User123 fez 3 logins acumulando um total de 120 minutos.
O User753 fez 3 logins acumulando um total de 15 minutos.
O User519 fez 4 logins acumulando um total de 111 minutos.
O User321 fez 2 logins acumulando um total de 189 minutos.
O User654 fez 2 logins acumulando um total de 100 minutos.
O User987 fez 1 logins acumulando um total de 25 minutos.
```

Utilizador(es) com maior número de Logins: "User519"